

ASSIGNMENT 2: FACTORY METHOD & ABSTRACT FACTORY

Student: Nursultan Maratov	Instructor: Yerassyl Bekenov
Group: SE-2523	Format: Java 17 Console Application
GitHub Repository: github.com/nursultanmaratov/logistics-assignment	Submitted Commit: ddee96e

1. INTRODUCTION & PATTERN RATIONALE

This project demonstrates the unified implementation of two creational design patterns in Java 17: **Factory Method** (Part A) and **Abstract Factory** (Part B). The system coordinates freight shipping logistics with cross-platform UI presentation under strict object-oriented principles.

1.1 Rationale for Factory Method in Logistics

Freight delivery requires polymorphic vehicle instantiation (road trucks vs. maritime container vessels). While each transportation mode exhibits unique operational behavior, the higher-level shipment workflow remains identical: plan cargo, instantiate carrier, and dispatch delivery. The **Factory Method** pattern defines an abstract creator (`Logistics`) declaring the abstract factory method `createTransport()` alongside a shared workflow (`planDelivery`). Subclasses (`RoadLogistics` and `SeaLogistics`) override the factory method to instantiate `Truck` and `Ship` respectively. This adheres to the *Open-Closed Principle*: new carriers can be introduced without modifying the core delivery algorithm.

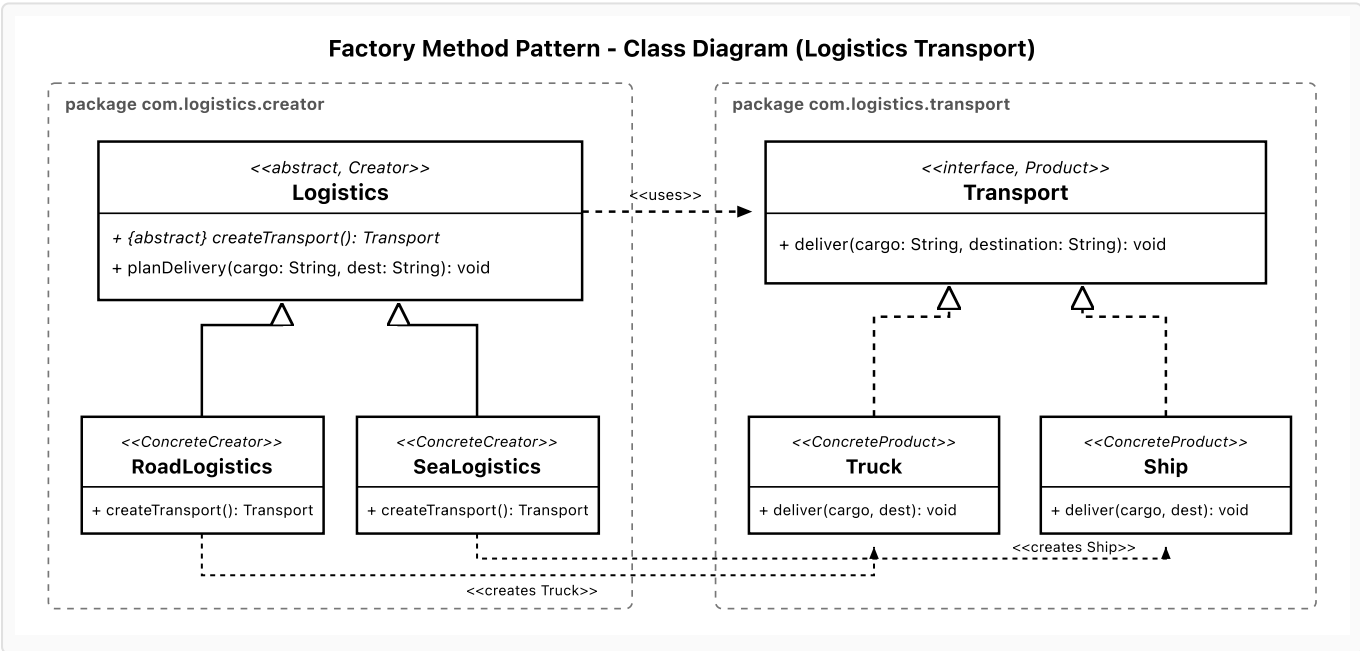
1.2 Rationale for Abstract Factory in UI Components

Graphical user interface elements exist in matching visual ecosystems (families). A button and checkbox presented to a user on Windows must not mix styling with macOS controls. The **Abstract Factory** pattern (`GUIFactory`) addresses this by declaring factory methods for both `Button` and `Checkbox`. Concrete factories (`WindowsFactory`, `MacOSFactory`) guarantee the creation of complete, matching component pairs. The client application (`DeliveryApplication`) accepts `GUIFactory` via constructor injection and interacts strictly with product interfaces without concrete coupling or type casting.

2. SYSTEM ARCHITECTURE & UML CLASS DIAGRAMS

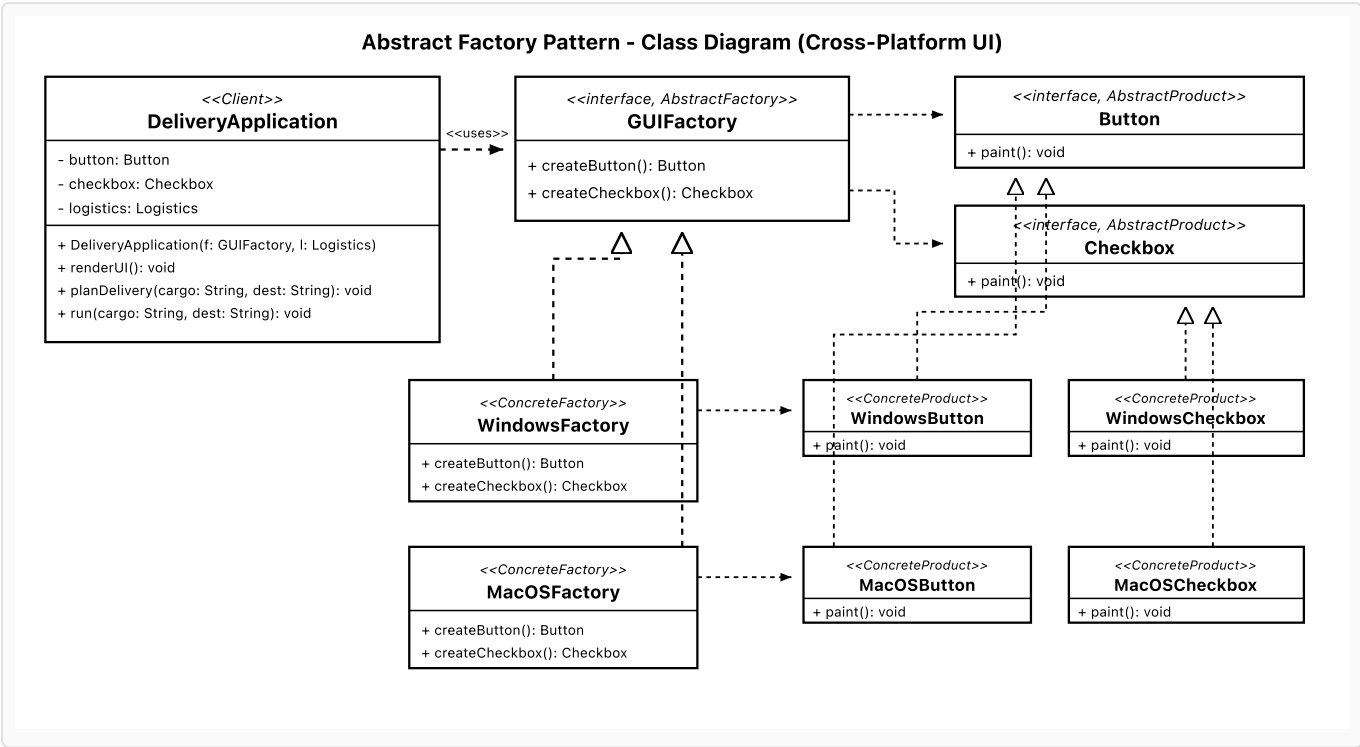
2.1 Factory Method Pattern Architecture

The creator hierarchy (`Logistics`) is completely separated from the product hierarchy (`Transport`). The template method `planDelivery()` accesses products exclusively through the contract defined by `createTransport()`.



2.2 Abstract Factory Pattern Architecture

The client (`DeliveryApplication`) depends solely on the abstract factory (`GUIFactory`) and product interfaces (`Button` , `Checkbox`). Concrete factories instantiate family-consistent product pairs.



3. CLEAN CODE EVIDENCE (SECTION 7)

Practice 1: Meaningful Names (Domain & Pattern Clarity)

Code Excerpt:

```
public class RoadLogistics extends Logistics {
    @Override
    public Transport createTransport() {
        return new Truck();
    }
}
```

Explanation & Benefit: Class, package, and method names explicitly convey their design pattern roles and business domains. `RoadLogistics` clearly indicates a concrete creator specializing in highway freight, while `createTransport()` explicitly declares its factory method responsibility. This avoids ambiguous names like `TransportHandler` or `ObjectManager`.

Practice 2: Small Methods (Single Responsibility Principle)

Code Excerpt:

```
public void renderUI() {
    button.paint();
    checkbox.paint();
}

public void planDelivery(String cargo, String destination) {
    logistics.planDelivery(cargo, destination);
}
```

Explanation & Benefit: Every method executes one discrete responsibility. UI rendering is completely separated from logistics planning. Adheres strictly to Robert C. Martin's maxim: *"Methods should do one thing. They should do it well. They should do it only."* This isolation simplifies unit testing and eliminates side effects.

Practice 3: Avoid Duplicated Logic (DRY Principle)

Code Excerpt:

```
public void planDelivery(String cargo, String destination) {
    Objects.requireNonNull(cargo, "Cargo cannot be null");
    Objects.requireNonNull(destination, "Destination cannot be null");
    Transport transport = createTransport();
    if (transport == null) {
        throw new IllegalStateException("Factory method returned null transport");
    }
    transport.deliver(cargo, destination);
}
```

Explanation & Benefit: Input validation, null-safety checks, factory invocation, and dispatch execution are implemented exactly once in the abstract `Logistics` class. Subclasses (`RoadLogistics`, `SeaLogistics`) are lightweight and contain zero duplicate orchestration logic.

Practice 4: Data Abstraction (Clean Code, Chapter 6)

Code Excerpt:

```
public class DeliveryApplication {
    private final Button button;
    private final Checkbox checkbox;
    private final Logistics logistics;

    public DeliveryApplication(GUIFactory factory, Logistics logistics) {
        this.button = Objects.requireNonNull(factory).createButton();
        this.checkbox = factory.createCheckbox();
        this.logistics = Objects.requireNonNull(logistics);
    }
}
```

Explanation & Benefit: Martin writes in Chapter 6: *"Hiding implementation is not just a matter of putting private variables between getters and setters. Rather, it is about abstractions."* The client holds pure interface references (`Button`, `Checkbox`, `Logistics`). It has zero knowledge of concrete implementations, underlying OS widgets, or transport vehicle mechanics.

Practice 5: Objects and Encapsulation (Clean Code, Chapter 6)

Code Excerpt:

```
public class Truck implements Transport {
    @Override
    public void deliver(String cargo, String destination) {
        Objects.requireNonNull(cargo, "Cargo must not be null");
        Objects.requireNonNull(destination, "Destination must not be null");
        System.out.println("Truck delivers " + cargo + " to " + destination);
    }
}
```

Explanation & Benefit: In Chapter 6, Martin distinguishes between procedural data structures (which expose state without behavior) and true objects (which hide state and expose behavior). Our product classes contain no anemic getters/setters; they express core domain behavior through verbs (`deliver`, `paint`), maintaining strict encapsulation.

4. VERIFICATION EVIDENCE (SECTION 6)

All six required runtime checks and missing input validation were tested and confirmed:

Check	Input Configuration	Expected Result	Actual Console Output	Status
1	ROAD + WINDOWS	Truck delivery; Windows button & checkbox	Rendering Windows button Rendering Windows checkbox Truck delivers laboratory equipment to Aktau warehouse	PASS
2	SEA + WINDOWS	Ship delivery; Windows button & checkbox	Rendering Windows button Rendering Windows checkbox Ship delivers laboratory equipment to Aktau warehouse	PASS
3	ROAD + MACOS	Truck delivery; macOS button & checkbox	Rendering macOS button Rendering macOS checkbox Truck delivers laboratory equipment to Aktau warehouse	PASS
4	SEA + MACOS	Ship delivery; macOS button & checkbox	Rendering macOS button Rendering macOS checkbox Ship delivers laboratory equipment to Aktau warehouse	PASS
5	AIR + WINDOWS (Invalid mode)	Clear validation message; no delivery executed	Validation Error: Unsupported delivery mode 'AIR'. Supported modes: ROAD, SEA.	PASS
6	ROAD + LINUX (Invalid platform)	Clear validation message; no UI construction	Validation Error: Unsupported UI platform	PASS

Check	Input Configuration	Expected Result	Actual Console Output	Status
			'Linux'. Supported platforms: WINDOWS, MacOS.	
7	[Missing Input] (Empty CLI args)	Clear validation message; clean program exit	Validation Error: Missing required configuration arguments.	PASS

Automated JUnit 5 Test Suite Verification:

```
[INFO] Running com.logistics.DeliveryApplicationTest -- Tests run: 5, Failures: 0, Errors: 0
[INFO] Running com.logistics.AbstractFactoryTest -- Tests run: 4, Failures: 0, Errors: 0
[INFO] Running com.logistics.ValidationTest -- Tests run: 13, Failures: 0, Errors: 0
[INFO] Running com.logistics.FactoryMethodTest -- Tests run: 5, Failures: 0, Errors: 0
[INFO] Results: Tests run: 27, Failures: 0, Errors: 0, Skipped: 0 | BUILD SUCCESS
```

5. CONCLUSION & DESIGN REFLECTION

5.1 Comparative Analysis

- **Simple Factory:** Relies on a centralized method containing conditional branching (if/switch). While straightforward for toy scenarios, it directly violates the Open-Closed Principle because adding new classes requires altering existing code.
- **Factory Method:** Leverages inheritance. An abstract creator declares a factory method and implements common workflow algorithms, deferring concrete product instantiation to subclasses. Ideal for managing variations of a single product family.
- **Abstract Factory:** Leverages object composition. Declares an interface for creating a suite of related products. Ensures consistent families (e.g. matching button and checkbox) without coupling clients to concrete classes.

5.2 Design Reflection: System Extensibility

1. **Adding a new Transport (e.g., Air Logistics):** Create Plane implements Transport and AirLogistics extends Logistics . Add AIR to DeliveryMode and update StartupHelper . Existing transport classes, creator subclasses, and DeliveryApplication remain 100% untouched.
2. **Adding a new UI Family (e.g., Linux platform):** Create LinuxButton , LinuxCheckbox , and LinuxFactory implements GUIFactory . Add LINUX to UIPlatform and update StartupHelper . Existing UI components and DeliveryApplication remain 100% untouched.
3. **Adding a new UI Product Type (e.g., TextField):** Define TextField interface. Modify GUIFactory to declare TextField createTextField() . Existing WindowsFactory and MacOSFactory must be modified to implement this method, and DeliveryApplication must be updated to invoke it. This illustrates the fundamental trade-off of Abstract Factory: adding new families is easy (OCP satisfied), but adding new product types is invasive.

6. REFERENCES

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

2. Freeman, E., & Robson, E. (2020). *Head First Design Patterns* (2nd ed.). O'Reilly Media. Chapter 4: The Factory Pattern.

3. Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall. Chapter 6: Objects and Data Structures.

4. Bekenov, Y. (2026). *Software Design Patterns - Lecture 2: Factory Method and Abstract Factory*. Astana IT University.